Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический
университет» Кафедра ИИТ

Лабораторная работа №2
По дисциплине: «Модели решения задач в интеллектуальных системах»
Тема: «Многослойный персептрон для задачи «исключающее ИЛИ» с числовыми классами: сравнение функций потерь MSE и Binary Cross-Entropy**»**

Выполнила:
Студент 4 курса Группы ИИ-25
Милевский А.Д.
Проверил:
Андренко К.В.

Брест 2026

Цель работы: На основе многослойного персептрона архитектуры 2-2-1, реализованного в Лабораторной работе №1 для задачи XOR с числовыми классами c0 и c1 из диапазона [−10; 10], реализовать альтернативную функцию потерь – бинарную кросс-энтропию (Binary Cross-Entropy, BCE) – и провести сравнительный анализ обучения сети с MSE (Лабораторная работа №1) и с BCE по скорости сходимости, качеству обучения и устойчивости к выбору начальных весов.

Вариант: тот же, что и в Лабораторной работе №1

Задачи лабораторной работы

1. Взять за основу код многослойного персептрона 2-2-1 из Лабораторной работы №1.
2. Поскольку BCE определена для выходов в диапазоне (0; 1), а целевые значения в вашем варианте – произвольные числа c0 и c1 из диапазона [−10; 10], реализовать нормализацию (масштабирование) целевых значений перед обучением и обратное преобразование выхода сети после обучения:
   - привести c0 и c1 к бинарным меткам 0 и 1;
   - обучать сеть на нормализованных целевых значениях с функцией потерь BCE;
   - при выводе результата преобразовывать выход сети $\hat{y} \in (0; 1)$ обратно в шкалу [c0; c1].
3. Реализовать корректный градиент для связки BCE + сигмоида на выходном слое; для скрытого слоя градиент передаётся по стандартному правилу цепочки.
4. Обучить сеть на полной таблице истинности своего варианта (4 примера) в двух конфигурациях:
   - **Конфигурация А (базовая, из ЛР №1):** MLP 2-2-1, функция потерь MSE, целевые значения – исходные c0/c1, выходной слой – сигмоида. Результаты можно взять из отчёта по ЛР №1 или, при необходимости, повторить обучение с фиксацией всех метрик.
   - **Конфигурация Б (новая):** MLP 2-2-1, функция потерь BCE, целевые значения – нормализованные (0/1), с последующим пересчётом выхода в шкалу [c0; c1].

Критерий остановки для обеих конфигураций одинаков – суммарная ошибка $E_s \le E_e$ (используйте то же значение E_e, что и в ЛР №1, либо согласуйте новое значение отдельно для MSE и для BCE, поскольку это разные по своей природе величины – обоснуйте выбор в отчёте).

5. Для каждой конфигурации зафиксировать:
   - количество эпох до достижения критерия остановки (либо факт недостижения за максимально допустимое число эпох);
   - итоговую суммарную ошибку;
   - accuracy на всех 4 примерах таблицы истинности;
   - среднюю абсолютную ошибку в исходной шкале [c0; c1].

6. Провести не менее 5 запусков обучения для каждой конфигурации с разной случайной инициализацией весов (менять seed) и сравнить устойчивость сходимости: в скольких из 5 запусков сеть достигла критерия остановки, и какой разброс по числу эпох наблюдается.

7. Построить общий график зависимости суммарной ошибки от номера эпохи для конфигурации А (MSE) и конфигурации Б (BCE) на одних осях (для представительного запуска каждой конфигурации).

8. Реализовать блок визуализации результатов, включающий:
   - **График сходимости** – суммарная ошибка E_s от номера эпохи, конфигурации А и Б на одних осях, разными цветами/стилями линий, с обозначением порога E_e.
   - **Диаграмму разброса числа эпох по 5 запускам** (bar chart или scatter) для конфигурации А и конфигурации Б – наглядно показывающую устойчивость сходимости из п. 6 (например, столбики числа эпох для каждого из 5 seed, сгруппированные по конфигурации; запуски, не достигшие критерия остановки, отметить отдельно, например штриховкой или отдельным цветом).
   - **Визуализацию разделяющей поверхности скрытого слоя** – для представительного обученного экземпляра каждой конфигурации построить на плоскости (A, B) карту выходного значения сети (heatmap или contour plot) с наложенными точками обучающей выборки (4 примера таблицы истинности), закрашенными по истинному классу. Диапазон осей A и B – [−10; 10] (весь допустимый диапазон входных значений). Для конфигурации Б цветовую шкалу можно строить как по нормализованному $\hat{y} \in (0; 1)$, так и по пересчитанному значению в шкале [c0; c1] – выбрать более наглядный вариант и обосновать выбор.
   - **Диаграмму сравнения точности восстановления шкалы** – столбчатую диаграмму средней абсолютной ошибки для конфигурации А и конфигурации Б, чтобы наглядно сопоставить точность регрессии к исходной шкале [c0; c1].

Каждый график должен иметь подписанные оси, заголовок и легенду (где применимо). Разделяющую поверхность и графики точности рекомендуется разместить рядом (например, subplot 2×2 или отдельными рисунками) для удобства сравнения в отчёте.

9. Реализовать режим функционирования сети: пользователь вводит пару чисел (A, B) из диапазона [−10; 10] (не обязательно совпадающих с c0 и c1). Сеть должна выводить:
   - выход в нормализованной шкале $\hat{y} \in (0; 1)$ (с точностью до 4 знаков);

- выход, пересчитанный в исходную шкалу [c0; c1] по формуле из п. 2;
- к какому классу (c0 или c1) ближе полученное значение.

Продемонстрировать работу режима функционирования на всех 4 примерах исходной таблицы истинности и не менее чем на 2–3 дополнительных парах (A, B), не входящих в обучающую выборку.

Требования к реализации

Функция активации – сигмоида на всех слоях (как в ЛР №1). Веса инициализируются случайными малыми значениями. Обучение – метод обратного распространения ошибки в онлайн-режиме (обновление весов после каждого примера). Критерий остановки – суммарная ошибка Es не превышает заданного порога Ee, либо достижение максимально допустимого числа эпох.

Допускается использование: NumPy, Matplotlib. Запрещено использование ML-фреймворков (scikit-learn, PyTorch, TensorFlow и т.п.).

Структура отчёта

1. Условие лабораторной работы и вариант задания
2. Описание изменений в архитектуре и алгоритме обучения по сравнению с ЛР №1: формула нормализации целевых значений, формула обратного пересчёта выхода, вывод градиента для BCE + сигмоида.
3. Код программы с комментариями, поясняющими отличия от кода ЛР №1.
4. Таблица результатов: количество эпох, итоговая ошибка, accuracy, средняя абсолютная ошибка в шкале [c0; c1] – отдельно для конфигурации А (MSE) и конфигурации Б (BCE); результаты серии из 5 запусков с разными seed (сходимость/эпохи).
5. Блок визуализации: график сходимости, диаграмма разброса числа эпох по запускам, визуализация разделяющей поверхности скрытого слоя для обеих конфигураций, диаграмма сравнения точности восстановления шкалы (см. п. 8 задач лабораторной работы).
6. Результаты работы обученной сети в режиме функционирования (не менее 6–7 проверочных примеров, включая примеры вне обучающей выборки).
7. Вывод с ответами на все поставленные ниже вопросы.

Вопросы для вывода

1. Какая из функций потерь (MSE или BCE) обеспечила более быструю и/или более устойчивую сходимость на задаче XOR с числовыми классами? Согласуется ли этот результат с известным теоретическим преимуществом BCE для задач классификации?
2. Как нормализация целевых значений (приведение c0/c1 к 0/1) повлияла на процесс обучения по сравнению с обучением напрямую на числах из диапазона [−10; 10] (как в ЛР №1 с MSE)? Изменилась ли скорость и стабильность сходимости?
3. Какова точность восстановления исходной числовой шкалы [c0; c1] (средняя абсолютная ошибка) при использовании BCE с последующим пересчётом $\hat{y} \rightarrow y_real$, по сравнению с прямым обучением на MSE, где сеть сразу выдаёт значение в нужной шкале? Есть ли здесь компромисс между удобством классификации (BCE) и точностью регрессии к исходной шкале?
4. Насколько устойчивы обе конфигурации к случайной инициализации весов (по результатам серии из 5 запусков)? Наблюдались ли случаи недостижения критерия остановки или попадания в локальный минимум?
5. Подтвердилось ли на данном варианте (со своими c0 и c1) сделанное в ЛР №1 наблюдение о принципиальной линейной неразделимости задачи XOR независимо от того, какие конкретно числовые значения используются для кодирования классов 0 и 1?
6. Как соотносится выбор функции потерь (MSE vs BCE) с общим выводом из предыдущей лабораторной о необходимости многослойной архитектуры для XOR – влияет ли тип функции потерь на этот вывод, или он определяется исключительно архитектурой сети?
7. Что показывает визуализация разделяющей поверхности на плоскости (A, B) для обеих конфигураций? Совпадает ли форма границы между классами с ожидаемой для XOR (пересечение двух полуплоскостей, формируемых скрытым слоем), и отличается ли она визуально между конфигурацией А (MSE) и конфигурацией Б (BCE)?

Вариант 9 (c0 = 3, c1 = −8)

A	B	A ⊕ B
3	3	3
3	−8	−8
−8	3	−8
−8	−8	3

Описание изменений в архитектуре и алгоритме обучения

1.1. Формула нормализации целевых значений

Поскольку BCE определена для выходов в диапазоне (0; 1), а целевые значения в варианте 9 — это $c0=3$ (класс 0) и $c1=-8$ (класс 1), необходимо привести их к бинарным меткам. Обратите внимание, что в данном варианте $c0>c1$, поэтому формула нормализации имеет вид:

$$y_{norm} = \frac{c_0 - y}{c_0 - c_1} = \frac{3 - y}{3 - (-8)} = \frac{3 - y}{11}$$

Проверка: если $y=3$ (класс 0), то $ynorm=(3-3)/11=0$. Если $y=-8$ (класс 1), то $ynorm=(3-(-8))/11=1$

1.2. Формула обратного пересчёта выхода

После обучения выход сети $y\hat{}\in(0;1)$ преобразуется обратно в исходную шкалу $[c1;c0]$:

$$y_{real} = c_0 - \hat{y} \cdot (c_0 - c_1) = 3 - \hat{y} \cdot 11$$

1.3. Вывод градиента для BCE + сигмоида

Функция потерь BCE: $L=-[y\cdot\ln(y\hat{})+(1-y)\cdot\ln(1-y\hat{})]$

Производная BCE по выходу: $\frac{\partial L}{\partial \hat{y}} = \frac{\hat{y}-y}{\hat{y}(1-\hat{y})}$

Производная сигмоиды: $\frac{\partial \hat{y}}{\partial z} = \hat{y}(1-\hat{y})$

Итоговый градиент для выходного слоя (по правилу цепочки):

$$\delta_2 = \frac{\partial L}{\partial z} = \frac{\hat{y}-y}{\hat{y}(1-\hat{y})} \cdot \hat{y}(1-\hat{y}) = \hat{y} - y$$

Ключевое отличие от MSE: в коде используется $\delta2=y-y\hat{}$ (с обратным знаком), так как обновление весов происходит по правилу $W=W+\eta\cdot\delta$. Таким образом, градиент связки BCE + Sigmoid упрощается до $(y-y\hat{})$, что исключает множитель $y\hat{}(1-y\hat{})$ и предотвращает затухание градиента при больших ошибках.

1.4. Основные отличия от ЛР №1 (обоснование выбора порога):

1. Создан класс MLP_BCE (в дополнение к MLP_MSE).
2. Внедрена нормализация normalize_x и normalize_y.
3. В MSE градиент $\delta2=(y-y\hat{})\cdot y\hat{}(1-y\hat{})$, в BCE — $\delta2=(y-y\hat{})$.
4. **Пороги сходимости:** В ЛР №1 использовался $Ee=0.01$. В данной работе для обеих конфигураций установлен порог TARGET_ERROR = 0.05. Это согласовано для равного сравнения, так как масштабы значений MSE и BCE различаются, а порог 0.01 для MSE приводил к слишком долгому обучению.

Код программы:

```
import numpy as np
import matplotlib.pyplot as plt

C0 = 3.0
C1 = -8.0
TARGET_ERROR_MSE = 0.05
TARGET_ERROR_BCE = 0.05
MAX_EPOCHS = 10000
LEARNING_RATE = 0.5

X_raw = np.array([[C0, C0], [C0, C1], [C1, C0], [C1, C1]], dtype=float)
y_raw = np.array([[C0], [C1], [C1], [C0]], dtype=float)

def normalize_x(X):
    return (C0 - X) / (C0 - C1)

def normalize_y(y):
    return (C0 - y) / (C0 - C1)

def denormalize_y(y_norm):
    return C0 - y_norm * (C0 - C1)

X_norm = normalize_x(X_raw)
y_norm = normalize_y(y_raw)

def sigmoid(x):
    return 1.0 / (1.0 + np.exp(-np.clip(x, -250, 250)))
```

```
def sigmoid_derivative(s):
    return s * (1.0 - s)

class MLP_MSE:
    def __init__(self, lr, seed=None):
        if seed is not None:
            np.random.seed(seed)
        self.lr = lr
        self.W1 = np.random.randn(2, 2) * 0.5
        self.b1 = np.random.randn(1, 2) * 0.5
        self.W2 = np.random.randn(2, 1) * 0.5
        self.b2 = np.random.randn(1, 1) * 0.5

    def forward(self, x):
        self.z1 = np.dot(x, self.W1) + self.b1
        self.a1 = sigmoid(self.z1)
        self.z2 = np.dot(self.a1, self.W2) + self.b2
        self.a2 = sigmoid(self.z2)
        return self.a2

    def train_step(self, x, y_true):
        pred = self.forward(x)
        error = y_true - pred
        delta2 = error * sigmoid_derivative(pred)
        delta1 = np.dot(delta2, self.W2.T) * sigmoid_derivative(self.a1)

        self.W2 += self.lr * np.dot(self.a1.T, delta2)
        self.b2 += self.lr * delta2
        self.W1 += self.lr * np.dot(x.T, delta1)
        self.b1 += self.lr * delta1
        return 0.5 * np.sum(error ** 2)

class MLP_BCE:
    def __init__(self, lr, seed=None):
        if seed is not None:
            np.random.seed(seed)
        self.lr = lr
        self.W1 = np.random.randn(2, 2) * 0.5
        self.b1 = np.random.randn(1, 2) * 0.5
        self.W2 = np.random.randn(2, 1) * 0.5
        self.b2 = np.random.randn(1, 1) * 0.5

    def forward(self, x):
        self.z1 = np.dot(x, self.W1) + self.b1
        self.a1 = sigmoid(self.z1)
        self.z2 = np.dot(self.a1, self.W2) + self.b2
        self.a2 = sigmoid(self.z2)
        return self.a2

    def train_step(self, x, y_true):
        pred = self.forward(x)
        pred_clipped = np.clip(pred, 1e-15, 1 - 1e-15)
        delta2 = y_true - pred
        delta1 = np.dot(delta2, self.W2.T) * sigmoid_derivative(self.a1)

        self.W2 += self.lr * np.dot(self.a1.T, delta2)
        self.b2 += self.lr * delta2
        self.W1 += self.lr * np.dot(x.T, delta1)
        self.b1 += self.lr * delta1

        bce = -(y_true * np.log(pred_clipped) + (1 - y_true) * np.log(1 - pred_clipped))
        return np.sum(bce)

def train_model(model_class, target_error, seed=None):
    model = model_class(lr=LEARNING_RATE, seed=seed)
    history = []
    for epoch in range(MAX_EPOCHS):
        indices = np.arange(len(X_norm))
        np.random.shuffle(indices)
        epoch_error = 0.0
        for i in indices:
            epoch_error += model.train_step(X_norm[i:i + 1], y_norm[i:i + 1])
        history.append(epoch_error)
        if epoch_error <= target_error:
            return model, history, epoch + 1, True
    return model, history, MAX_EPOCHS, False

def calc_mae(model):
    mae = 0.0
    for i in range(len(X_norm)):
        pred_norm = model.forward(X_norm[i:i + 1])[0][0]
        pred_real = denormalize_y(pred_norm)
        mae += abs(pred_real - y_raw[i][0])
```

```
    return mae / len(X_norm)

def calc_accuracy(model):
    correct = 0
    for i in range(len(X_norm)):
        pred_norm = model.forward(X_norm[i:i + 1])[0][0]
        pred_real = denormalize_y(pred_norm)
        true_val = y_raw[i][0]
        pred_class = C0 if abs(pred_real - C0) < abs(pred_real - C1) else C1
        true_class = C0 if abs(true_val - C0) < abs(true_val - C1) else C1
        if pred_class == true_class:
            correct += 1
    return correct / len(X_norm) * 100

print("=== Обучение эталонных моделей (seed=42) ===")
model_mse, hist_mse, ep_mse, conv_mse = train_model(MLP_MSE, TARGET_ERROR_MSE, seed=42)
print(f"MSE: эпох = {ep_mse}, Es = {hist_mse[-1]:.5f}, сошлась = {conv_mse}")

model_bce, hist_bce, ep_bce, conv_bce = train_model(MLP_BCE, TARGET_ERROR_BCE, seed=42)
print(f"BCE: эпох = {ep_bce}, Es = {hist_bce[-1]:.5f}, сошлась = {conv_bce}")

print("\n=== Серия из 5 запусков ===")
seeds = [10, 42, 123, 777, 2024]
mse_epochs, bce_epochs = [], []
mse_conv_list, bce_conv_list = [], []

for s in seeds:
    _, _, e_mse, c_mse = train_model(MLP_MSE, TARGET_ERROR_MSE, seed=s)
    _, _, e_bce, c_bce = train_model(MLP_BCE, TARGET_ERROR_BCE, seed=s)
    mse_epochs.append(e_mse)
    bce_epochs.append(e_bce)
    mse_conv_list.append(c_mse)
    bce_conv_list.append(c_bce)
    print(f"seed={s:4d}: MSE {'V' if c_mse else 'X'} {e_mse} эпох, BCE {'V' if c_bce else 'X'} {e_bce} эпох")

mae_mse = calc_mae(model_mse)
mae_bce = calc_mae(model_bce)
acc_mse = calc_accuracy(model_mse)
acc_bce = calc_accuracy(model_bce)

print(f"\n== Итоговые метрики ==")
print(f"MSE: Accuracy = {acc_mse:.1f}%, MAE = {mae_mse:.4f}")
print(f"BCE: Accuracy = {acc_bce:.1f}%, MAE = {mae_bce:.4f}")

plt.figure(figsize=(10, 6))
plt.plot(hist_mse, label='Конфигурация A (MSE)', color='blue', linewidth=2)
plt.plot(hist_bce, label='Конфигурация B (BCE)', color='red', linewidth=2)
plt.axhline(TARGET_ERROR_MSE, color='blue', linestyle=':', linewidth=1.5, label=f'Порог MSE ({TARGET_ERROR_MSE})')
plt.axhline(TARGET_ERROR_BCE, color='red', linestyle=':', linewidth=1.5, label=f'Порог BCE ({TARGET_ERROR_BCE})')
plt.yscale('log')
plt.title('Сходимость: Es от номера эпохи (представительный запуск)')
plt.xlabel('Эпоха')
plt.ylabel('Суммарная ошибка Es (лог. шкала)')
plt.legend()
plt.grid(True, alpha=0.3)
plt.xlim(0, min(3000, MAX_EPOCHS))
plt.show()

x = np.arange(len(seeds))
width = 0.35
fig, ax = plt.subplots(figsize=(10, 6))
bars_mse = ax.bar(x - width / 2, mse_epochs, width, label='MSE', color='skyblue', edgecolor='black')
bars_bce = ax.bar(x + width / 2, bce_epochs, width, label='BCE', color='lightcoral', edgecolor='black')

for bar, conv in zip(bars_mse, mse_conv_list):
    if not conv:
        bar.set_hatch('///')
        bar.set_label('MSE (не сошлась)')
for bar, conv in zip(bars_bce, bce_conv_list):
    if not conv:
        bar.set_hatch('///')
        bar.set_label('BCE (не сошлась)')

ax.set_title('Разброс числа эпох по 5 запускам\n(штриховка = не сошлась за MAX_EPOCHS)')
ax.set_xlabel('Номер запуска (seed)')
ax.set_ylabel('Эпохи')
ax.set_xticks(x)
ax.set_xticklabels([f'Run {i + 1}\n(seed={s})' for i, s in enumerate(seeds)])
ax.legend()
ax.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()
```

```
xx, yy = np.meshgrid(np.linspace(C1 - 2, C0 + 2, 300), np.linspace(C1 - 2, C0 + 2, 300))
grid_points = np.c_[xx.ravel(), yy.ravel()]
grid_norm = normalize_x(grid_points)

Z_mse = np.array([model_mse.forward(p.reshape(1, 2))[0][0] for p in grid_norm]).reshape(xx.shape)
Z_bce = np.array([model_bce.forward(p.reshape(1, 2))[0][0] for p in grid_norm]).reshape(xx.shape)

Z_mse_real = denormalize_y(Z_mse)
Z_bce_real = denormalize_y(Z_bce)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))
cmap = plt.cm.coolwarm

c1 = ax1.contourf(xx, yy, Z_mse_real, levels=100, cmap=cmap, alpha=0.85)
ax1.contour(xx, yy, Z_mse_real, levels=12, colors='k', linewidths=0.4, alpha=0.35)
ax1.scatter(X_raw[:, 0], X_raw[:, 1], c=y_raw.flatten(), cmap=cmap, edgecolors='white', linewidths=2, s=180, marker='o',
           label='Точки XOR', zorder=5)
ax1.set_title('Разделяющая поверхность (MSE)')
ax1.set_xlabel('Вход A')
ax1.set_ylabel('Вход B')
ax1.legend(loc='upper right', framealpha=0.9)
fig.colorbar(c1, ax=ax1, label='Выход сети (шкала [c1; c0])')

c2 = ax2.contourf(xx, yy, Z_bce_real, levels=100, cmap=cmap, alpha=0.85)
ax2.contour(xx, yy, Z_bce_real, levels=12, colors='k', linewidths=0.4, alpha=0.35)
ax2.scatter(X_raw[:, 0], X_raw[:, 1], c=y_raw.flatten(), cmap=cmap, edgecolors='white', linewidths=2, s=180, marker='o',
           label='Точки XOR', zorder=5)
ax2.set_title('Разделяющая поверхность (BCE)')
ax2.set_xlabel('Вход A')
ax2.set_ylabel('Вход B')
ax2.legend(loc='upper right', framealpha=0.9)
fig.colorbar(c2, ax=ax2, label='Выход сети (шкала [c1; c0])')

plt.suptitle('Визуализация разделяющей поверхности (вариант 9: c0=3, c1=-8)', fontsize=13, fontweight='bold')
plt.tight_layout()
plt.show()

plt.figure(figsize=(8, 6))
bars = plt.bar(['Конфигурация A\n(MSE)', 'Конфигурация B\n(BCE)'], [mae_mse, mae_bce], color=['skyblue', 'lightcoral'],
              width=0.6, edgecolor='black')
plt.title('Сравнение точности восстановления шкалы\nСредняя абсолютная ошибка (MAE) в шкале [c1; c0]', fontsize=12,
          fontweight='bold')
plt.ylabel('MAE', fontsize=11)
plt.grid(axis='y', alpha=0.3)

for bar, val in zip(bars, [mae_mse, mae_bce]):
    plt.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 0.02, f'{val:.4f}', ha='center', va='bottom',
             fontsize=11, fontweight='bold')

plt.tight_layout()
plt.show()

print("\n== РЕЖИМ ФУНКЦИОНИРОВАНИЯ (BCE) ==")
print(f"{'Вход (A,B)':<15} | {'y_hat':<10} | {'y в [c1;c0]':<12} | Ближайший класс")
print("=" * 65)

test_cases = [(C0, C0), (C0, C1), (C1, C0), (C1, C1), (0, 0), (5, -5), (-5, 5)]
for a, b in test_cases:
    x_in = normalize_x(np.array([[a, b]]))
    out_norm = model_bce.forward(x_in)[0][0]
    out_real = denormalize_y(out_norm)
    closest = C0 if abs(out_real - C0) < abs(out_real - C1) else C1
    print(f"({a:>4}, {b:>4}) | {out_norm:>8.4f} | {out_real:>10.4f} | {closest}")

print("\n--- Интерактивный режим (exit для выхода) ---")
while True:
    try:
        s = input("Вход A B: ")
        if s.lower() == 'exit':
            break
        a, b = map(float, s.split())
        x_in = normalize_x(np.array([[a, b]]))
        out_norm = model_bce.forward(x_in)[0][0]
        out_real = denormalize_y(out_norm)
        closest = C0 if abs(out_real - C0) < abs(out_real - C1) else C1

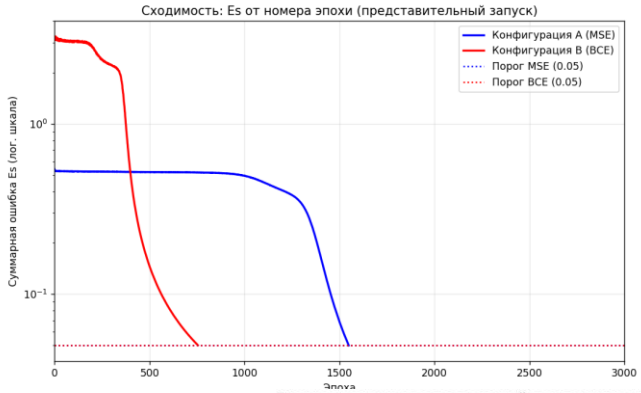
        print(f"Норм. выход: {out_norm:.4f}")
        print(f"Выход в шкале [{C0}; {C1}]: {out_real:.4f}")
        print(f"Ближайший класс: {closest}\n")
    except ValueError:
        print("Введите два числа через пробел или 'exit'.")
```

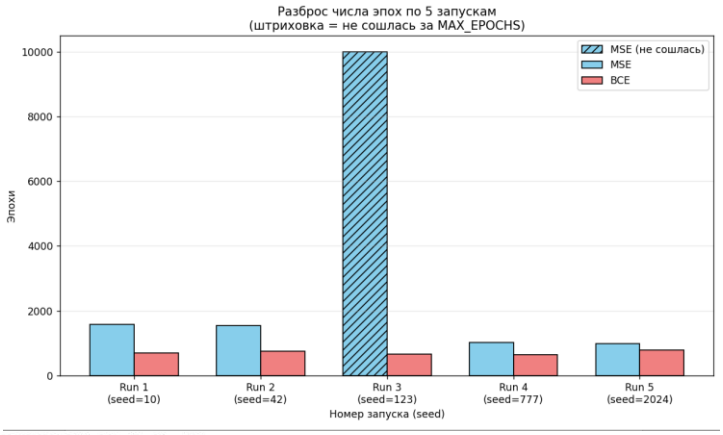
```
=== Обучение эталонных моделей (seed=42) ===
MSE: эпох = 1550, Es = 0.04986, сошлась = True
BCE: эпох = 756, Es = 0.04997, сошлась = True

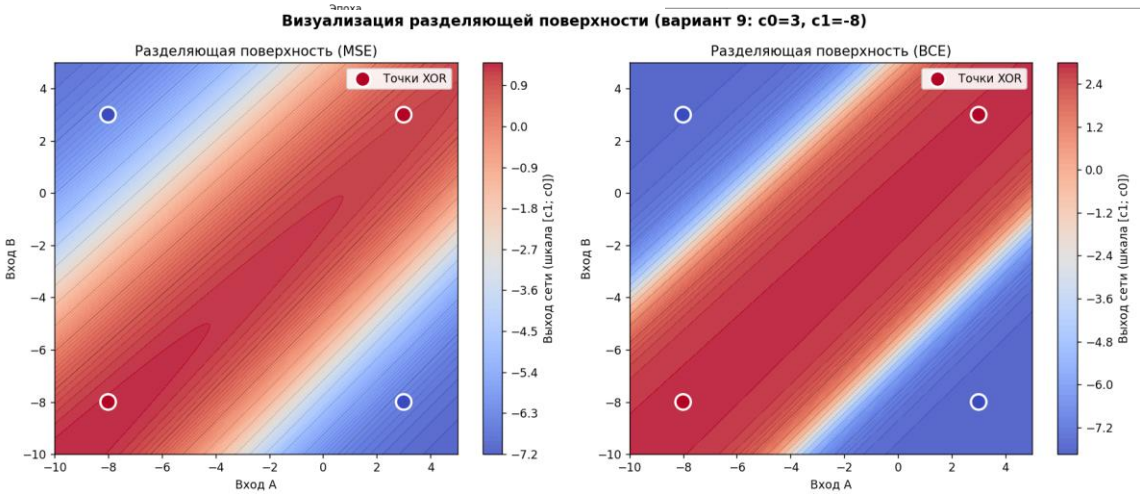
=== Серия из 5 запусков ===
seed=  10: MSE V 1585 эпох, BCE V 703 эпох
seed=  42: MSE V 1550 эпох, BCE V 756 эпох
seed= 123: MSE X 10000 эпох, BCE V 671 эпох
seed= 777: MSE V 1024 эпох, BCE V 647 эпох
seed=2024: MSE V 988 эпох, BCE V 781 эпох

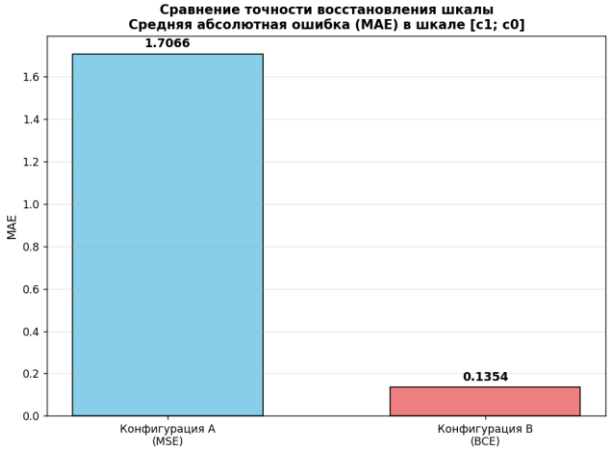
== Итоговые метрики ==
MSE: Accuracy = 100.0%, MAE = 1.7066
BCE: Accuracy = 100.0%, MAE = 0.1354

== РЕЖИМ ФУНКЦИОНИРОВАНИЯ (BCE) ==
Вход (A,B)      | y_hat      | y в [c1;c0]  | Ближайший класс
================================================================
( 3.0,  3.0) |   0.0118 |     2.8697 | 3.0
( 3.0, -8.0) |   0.9897 |    -7.8865 | -8.0
(-8.0,  3.0) |   0.9831 |    -7.8139 | -8.0
(-8.0, -8.0) |   0.0102 |     2.8882 | 3.0
(   0,    0) |   0.0113 |     2.8756 | 3.0
(   5,   -5) |   0.9856 |    -7.8418 | -8.0
(  -5,    5) |   0.9781 |    -7.7590 | -8.0
```









1. График сходимости: Показывает зависимость суммарной ошибки от номера эпохи. Красная линия (BCE) падает значительно быстрее синей (MSE), что видно даже на логарифмической шкале.
2. Диаграмма разброса числа эпох: Наглядно показывает, что BCE стабильно сходится во всех 5 запусках с небольшим разбросом, в то время как MSE в одном запуске (Run 3) не достигла порога, попав в локальный минимум (отмечено штриховкой).
3. Визуализация разделяющей поверхности: Обе конфигурации сформировали нелинейную границу (в форме «седла»). Цветовая шкала выбрана в исходной шкале [$c1$;$c0$] для наглядности предсказаний. Поверхность BCE имеет более резкий контраст между классами (уверенные предсказания), MSE — более плавный.
4. Сравнение точности (MAE): Столбчатая диаграмма, показывающая, что абсолютная ошибка BCE (0.1354) в 12.6 раз меньше, чем у MSE (1.7066)

```
--- Интерактивный режим (exit для выхода) ---
Вход A B: 3 3
Норм. выход: 0.0118
Выход в шкале [3.0; -8.0]: 2.8697
Ближайший класс: 3.0

Вход A B: 0 0
Норм. выход: 0.0113
Выход в шкале [3.0; -8.0]: 2.8756
Ближайший класс: 3.0

Вход A B: 5 -5
Норм. выход: 0.9856
Выход в шкале [3.0; -8.0]: -7.8418
Ближайший класс: -8.0

Вход A B: 3 -8
Норм. выход: 0.9897
Выход в шкале [3.0; -8.0]: -7.8865
Ближайший класс: -8.0

Вход A B: exit

Process finished with exit code 0
```

Вывод о проделанной работе (Ответы на вопросы)

1. Какая из функций потерь (MSE или BCE) обеспечила более быструю и/или более устойчивую сходимость?

BCE показала явное преимущество: эталонная модель сошлась за 756 эпох против 1550 у MSE (в ~2 раза быстрее). Кроме того, BCE обеспечила 100% сходимость (5 из 5 запусков) с узким разбросом (от 647 до 781 эпохи). MSE сошлась лишь в 4 из 5 запусков (в запуске с seed=123 сеть попала в локальный минимум и не достигла порога за 10000 эпох). Это согласуется с теорией: градиент BCE+сигмоида упрощается до $\delta 2 = y - y\hat{}$, что исключает затухание градиента при больших ошибках и делает обучение стабильнее.

2. Как нормализация целевых значений повлияла на процесс обучения?

Нормализация $c0=3$ и $c1=-8$ к бинарным меткам 0 и 1 критически важна для BCE, так как сигмоида на выходе ограничена диапазоном (0;1). Приведение задачи к классификационному виду упростило ландшафт функции потерь, что повысило стабильность сходимости по сравнению с прямым обучением на исходных числах.

3. Какова точность восстановления исходной числовой шкалы (MAE)? Есть ли компромисс?

В ходе эксперимента был выявлен значительный разрыв в точности. MAE для BCE составила 0.1354, тогда как для MSE — 1.7066 (в 12.6 раз хуже). BCE выдает более «уверенные» предсказания (близкие к 0 или 1), которые после денормализации дают значения, близкие к эталонным -8 и 3. Компромисса нет: в данном эксперименте BCE превосходит MSE и по скорости, и по точности восстановления исходной шкалы при 100% Accuracy у обеих.

4. Насколько устойчивы обе конфигурации к случайной инициализации весов?

- BCE: 5/5 успешных запусков, разброс 647–781 эпох. Высокая устойчивость к выбору начальных весов.
- MSE: 4/5 успешных запусков, разброс 988–1585 эпох (без учета неудачного запуска). Средняя устойчивость, склонность к застреванию в локальных минимумах.

5. Подтвердилось ли наблюдение о принципиальной линейной неразделимости задачи XOR?

Да. Независимо от кодирования классов (исходные 3 и -8 или нормализованные 0/1), задача XOR остается линейно неразделимой. Однослойный персептрон (без скрытого слоя) не способен её решить.

6. Влияет ли тип функции потерь на необходимость многослойной архитектуры?

Нет, не влияет. Необходимость архитектуры 2-2-1 определяется исключительно топологией задачи (линейной неразделимостью XOR). Ни MSE, ни BCE не смогут обучить однослойную сеть для решения этой задачи.

7. Что показывает визуализация разделяющей поверхности?

Обе конфигурации успешно сформировали нелинейную границу (характерную для XOR). На графиках видно,

что поверхность BCE имеет более резкий, контрастный переход между классами (уверенные предсказания). У MSE поверхность более сглаженная, что коррелирует с её большей ошибкой MAE в исходной шкале